CS527 – Lab 8: OpenMP Report (Search, Sort, Histogram, Transpose, Convolution)

Name: Soudip Das · Roll no.: 2026CSM1037

1. Setup and method

Item	Value
Language	C++ (standard C++ with OpenMP)
Compiler	g++ from MSYS2 UCRT64 (Windows)
Compile command	`g++ -fopenmp -O2 -Wall taskN_xxx.cpp -o taskN` (no warnings)
Machine	[CPU model : Intel(R) Core(TM) Ultra 5 225H (1.70 GHz)], [RAM : 32.0 GB (31.4 GB usable)]. OpenMP reports 14 hardware threads (omp_get_max_threads() = 14)
Timing	`omp_get_wtime()`
Setting the number of threads	**In code, with `omp_set_num_threads(n)`**. Each program loops over 1, 2, 4 and 8 threads itself
Correctness	Every parallel version is compared with a serial version before its time is reported. The Match / Correct columns are all **YES** (except the deliberately broken naive histogram in Task 3)
Data layout	Tasks 4 and 5 use flattened 1D vectors: element (i, j) is `A[i * n + j]`

Speedup = T_serial / T_parallel. **Efficiency** = speedup / number of threads.

2. Task 1 – Array search

What was done. Search an array of random integers for a key. The serial version loops and stops (`break`) at the first match. OpenMP does not allow `break` in a `parallel for` , so the parallel version checks every element and uses `reduction(min:first)` to keep the smallest matching index. This returns the first occurrence, exactly like the serial search, and gives -1 if the key is absent. The key was placed at 10% of the array, in the middle, at the very end, twice (at 50% and 75%), and not at all. All 40 index comparisons with the serial result: **Match = YES**.

Each cell below is **parallel time (ms) / speedup**.

N = 10^7

Case	Serial (ms)	1 thread	2 threads	4 threads	8 threads	Index found
key at 10%	0.200	3.100 / 0.06	1.850 / 0.11	1.100 / 0.18	1.100 / 0.18	1000000
key in the middle	2.000	3.150 / 0.63	1.900 / 1.05	1.300 / 1.54	0.850 / 2.35	5000000
key at the very end	4.000	2.950 / 1.36	1.900 / 2.11	1.050 / 3.81	0.950 / 4.21	9999999
key twice (50%, 75%)	2.000	3.000 / 0.67	1.850 / 1.08	1.100 / 1.82	0.900 / 2.22	5000000
key not present	4.050	3.100 / 1.31	1.700 / 2.38	1.150 / 3.52	0.900 / 4.50	-1

N = 10^8

Case	Serial (ms)	1 thread	2 threads	4 threads	8 threads	Index found
key at 10%	4.000	26.200 / 0.15	16.800 / 0.24	11.400 / 0.35	8.200 / 0.49	10000000
key in the middle	18.800	26.000 / 0.72	17.000 / 1.11	11.200 / 1.68	8.400 / 2.24	50000000
key at the very end	37.800	24.800 / 1.52	17.200 / 2.20	11.200 / 3.38	8.200 / 4.61	99999999
key twice (50%, 75%)	18.800	25.400 / 0.74	16.800 / 1.12	11.400 / 1.65	8.000 / 2.35	50000000
key not present	37.400	28.800 / 1.30	17.400 / 2.15	12.000 / 3.12	8.200 / 4.56	-1

Explanation of the speedup. The parallel search always scans the whole array, while the serial search stops at the first match. So when the key is early (10%, middle) the serial code does little work and the parallel code is slower or only slightly faster (0.49 and 2.24 on 8 threads at N = 10^8). When the whole array must be scanned (key at the end, or missing), the parallel search gives the best speedups, about 4.6 on 8 threads. This is below 8 because scanning an array is limited by memory bandwidth. The parallel loop on one thread is also faster than the serial full scan (24.8 ms against 37.8 ms), probably because a

loop without `break` is easier for the compiler to speed up, so part of the speedup in these rows is not due to threads. The case with the key twice returns index 5000000, the first of the two positions, which confirms that the smallest index is returned.

3. Task 2 – Array sorting

What was done. Bubble sort cannot be parallelised directly, because each comparison depends on the previous one. The parallel version is **odd-even transposition sort**: in even phases the pairs (0,1), (2,3), ... are compared and swapped, in odd phases the pairs (1,2), (3,4), ... The N phases sort the array. Pairs inside one phase are independent, so threads can handle them at the same time. The results were compared with `std::sort`: **all sorted correctly (YES)**. Three serial references are reported: bubble sort, the same odd-even algorithm without OpenMP, and `std::sort`. Odd-even sort is a different algorithm from bubble sort, so the speedup against the serial odd-even sort is the fair measure of OpenMP itself.

Synchronization required.

1. Inside a phase no synchronization is needed. Each thread swaps its own pairs, and no two pairs share an element, so there is no data race.
2. Between phases a **barrier** is required: phase p+1 compares elements that phase p may have just swapped, so every thread must finish phase p first. The barrier is the implicit one at the end of each `#pragma omp for`. The threads are created once (`#pragma omp parallel`) with the `omp for` inside the phase loop, instead of creating them N times.

N = 10000 (serial bubble 0.0280 s, serial odd-even 0.0240 s, `std::sort` about 0 s)

Threads	Parallel (s)	Speedup vs bubble	Speedup vs serial odd-even	Efficiency	Correct
1	0.0240	1.17	1.00	1.00	YES
2	0.0860	0.33	0.28	0.14	YES
4	0.1680	0.17	0.14	0.04	YES
8	0.3330	0.08	0.07	0.01	YES

N = 20000 (serial bubble 0.1680 s, serial odd-even 0.0930 s, `std::sort` 0.0010 s)

Threads	Parallel (s)	Speedup vs bubble	Speedup vs serial odd-even	Efficiency	Correct
1	0.0920	1.83	1.01	1.01	YES
2	0.2380	0.71	0.39	0.20	YES

Threads	Parallel (s)	Speedup vs bubble	Speedup vs serial odd-even	Efficiency	Correct
4	0.3720	0.45	0.25	0.06	YES
8	0.7340	0.23	0.13	0.02	YES

N = 40000 (serial bubble 1.2450 s, serial odd-even 0.3540 s, `std::sort` 0.0020 s)

Threads	Parallel (s)	Speedup vs bubble	Speedup vs serial odd-even	Efficiency	Correct
1	0.3720	3.35	0.95	0.95	YES
2	0.6480	1.92	0.55	0.27	YES
4	0.7970	1.56	0.44	0.11	YES
8	1.5380	0.81	0.23	0.03	YES

Explanation of the speedup (or lack of it). There is no real speedup: the OpenMP odd-even sort is slower than the serial odd-even sort at every size once more than one thread is used, and gets slower as threads are added. The apparent speedups against bubble sort at 1 or 2 threads come from the algorithm (its comparisons are independent), not from threads. The reason is the barrier: there are N phases, and each phase has only N/2 comparisons. At N = 40000 one phase takes about 9 µs of work on 1 thread, but 1.5380 s / 40000 phases is about 38 µs per phase on 8 threads. The cost of synchronizing the threads in every phase is larger than the work saved. The speedup against serial odd-even improves with N (0.07, 0.13, 0.23 on 8 threads), because the work per phase grows while the barrier cost stays the same, but within these sizes it never reaches 1. Both algorithms are O(N^2); `std::sort` is thousands of times faster than any of them.

4. Task 3 – Histogram calculation

What was done. Count how often each value 0 to 9 appears in a random array. The race: `hist[a[i]]++` is a read, an add and a write, so when two threads increment the same bin at once, one update is lost. Five parallel versions were tried:

- **Naive**: `hist[a[i]]++` with no protection (broken, shown only to demonstrate the race).
- **atomic**: `#pragma omp atomic` makes each increment indivisible.
- **critical**: `#pragma omp critical` lets one thread at a time do the increment.
- **Local copy**: each thread counts into its own private 10-element array, then adds it to the shared histogram once at the end (inside a `critical` block).

- **reduction**: `reduction(+:hist[0:10])`, which does the private-copies idea automatically.

All four correct versions produced the same histogram as the serial code in every run (**Correct = YES**).

Race demonstration (N = 10^7, 4 threads, first run)

Value	0	1	2	3	4	5	6	7	8	9	Total
Serial (correct)	1000266	1002109	1000716	998675	1001187	998726	997168	999349	1000924	1000880	10000000
Naive run 1	266217	265963	266765	263743	265766	263255	264349	261607	263378	264984	2646027
Naive run 2	276946	277123	277049	277890	277785	277609	277143	275967	276390	277512	2771414
Naive run 3	266935	265271	265383	263934	266844	266237	264354	265448	267350	266907	2658663

The naive version loses most of the counts: only about a quarter of the 10,000,000 increments survived (2.4 to 2.8 million in all nine naive runs across the repeated executions), and the answer changes from run to run.

Timing, N = 10^7 (serial 0.005 s). Each cell is **time (s) / speedup**.

Threads	Atomic	Critical	Local copy	Reduction
1	0.0440 / 0.11	0.1300 / 0.04	0.0050 / 1.00	0.0050 / 1.00
2	0.1430 / 0.03	0.3880 / 0.01	0.0020 / 2.50	0.0030 / 1.67
4	0.1630 / 0.03	0.3200 / 0.02	0.0010 / 5.00	0.0010 / 5.00
8	0.1730 / 0.03	0.4810 / 0.01	0.0010 / 5.00	0.0010 / 5.00

At N = 10^7 the fast versions take only 1 to 5 timer ticks, so their speedups are rough.

Timing, N = 10^8 (serial 0.048 s)

Threads	Atomic	Critical	Local copy	Reduction
1	0.4240 / 0.11	1.1970 / 0.04	0.0480 / 1.00	0.0490 / 0.98
2	1.4020 / 0.03	4.1520 / 0.01	0.0250 / 1.92	0.0260 / 1.85
4	1.5890 / 0.03	3.2390 / 0.01	0.0170 / 2.82	0.0140 / 3.43

Threads	Atomic	Critical	Local copy	Reduction
8	1.7180 / 0.03	4.7850 / 0.01	0.0080 / 6.00	0.0070 / 6.86

Explanation of the speedup. The local-copy and reduction versions scale well (6.0 and 6.9 on 8 threads at N = 10^8, efficiency 0.75 and 0.86; in the repeated runs 5.7 to 6.4 and 6.4 to 6.9), because threads never touch shared data while counting and combine only 10 numbers at the end. `atomic` and `critical` are correct but slower than the serial loop at every thread count, and get slower with more threads (atomic is about 9 times slower than serial on 1 thread and about 36 times on 8; critical about 25 and 100 times). Every element update has to wait for the same 10 shared counters, so the threads queue up and the counters bounce between the cores' caches.

How the race is prevented. Either make each update indivisible or serialized (`atomic` , `critical` , which are correct but slow), or, better, avoid sharing during the loop by giving each thread private counters and merging once (local copy, `reduction`).

5. Task 4 – Matrix transpose

What was done. B[j][i] = A[i][j], with matrices as flattened 1D vectors. Versions: serial; parallel on the outer loop only; `collapse(2)` ; and a bonus **blocked** version that works on 32 x 32 tiles, so both the rows read and the columns written stay in the cache. There is no race: each element of A is copied to a different element of B. All versions matched the serial transpose (**Correct = YES** on every row). Times are averages in milliseconds. Each cell is **time (ms) / speedup**.

N = 1024 (serial 3.957 ms)

Threads	Outer	Collapse(2)	Blocked
1	4.191 / 0.94	4.170 / 0.95	6.830 / 0.58
2	2.277 / 1.74	2.106 / 1.88	3.553 / 1.11
4	1.234 / 3.21	1.191 / 3.32	1.851 / 2.14
8	1.085 / 3.65	0.936 / 4.23	0.957 / 4.13

N = 4096 (serial 210.000 ms)

Threads	Outer	Collapse(2)	Blocked
1	135.500 / 1.55	147.000 / 1.43	163.000 / 1.29
2	67.000 / 3.13	75.500 / 2.78	85.500 / 2.46
4	43.000 / 4.88	45.000 / 4.67	44.000 / 4.77
8	30.000 / 7.00	29.000 / 7.24	20.500 / 10.24

N = 8192 (serial 774.000 ms)

Threads	Outer	Collapse(2)	Blocked
1	741.000 / 1.04	740.000 / 1.05	547.000 / 1.41
2	381.000 / 2.03	392.000 / 1.97	286.000 / 2.71
4	212.000 / 3.65	218.000 / 3.55	149.000 / 5.19
8	149.000 / 5.19	151.000 / 5.13	84.000 / 9.21

Explanation of the speedup. Speedup grows with the number of threads and with the matrix size: for N = 8192 the outer-loop version reaches 5.2 on 8 threads (efficiency 0.65), while for N = 1024 the matrix is small and the speedup is only 3.7 to 4.2. Transpose does no arithmetic and reads one way (along rows) while writing the other way (down columns), so it is limited by memory access; this explains why it stays below 8. `collapse(2)` makes no consistent difference, since even N = 1024 gives plenty of rows to share. The blocked version is the best for large matrices (9.2 and 10.2 on 8 threads); a value above 8 is possible because it is also better for the cache than the serial code it is compared with (it is already 1.3 to 1.4 times faster on one thread), but for N = 1024 it is slower on one thread (0.58) because the extra tile loops cost more than they save when the matrix already fits in the cache. A second run gave the same picture (outer 5.25 to 5.45, blocked 9.8 to 9.9 on 8 threads for N = 4096 and 8192), but individual numbers varied a lot, for example outer with 1 thread at N = 4096 took 135.5 ms in one run and 278 ms in the other.

6. Task 5 – 2D convolution

What was done. A 3x3 filter slides over the image. At each position the 9 pixels under the filter are multiplied by the 9 filter numbers and summed. Only positions where the filter fits fully are used (no padding), so an N x N image gives an (N-2) x (N-2) output. The image and output are flattened 1D vectors; output pixel (i, j) uses image pixels (i..i+2, j..j+2). Every output pixel is computed independently: threads only read the image and each writes its own output pixels, so there is no race and no lock. Two parallel versions were written: the outer loop only, and `collapse(2)`. Each thread keeps its own copy of the 9 filter values.

Check with the 5x5 example from the assignment. With the averaging filter (all weights 1/9) the program printed the 3 x 3 output

70.00	80.00	90.00
120.00	130.00	140.00
170.00	180.00	190.00

The first value is 630/9 = 70, as worked out in the assignment. The parallel result matched the serial one for this filter and for two others tried on the same image (a sharpen filter and an edge-detection filter). On this smooth image the sharpen filter leaves the values unchanged and the edge filter gives all zeros, as expected.

Timing with the averaging filter. Each cell is **time (ms) / speedup**; every result matched the serial output (**Correct = YES**).

Image 1024 x 1024 (serial 1.468 ms)

Threads	Outer	Collapse(2)
1	1.489 / 0.99	1.723 / 0.85
2	0.766 / 1.92	0.936 / 1.57
4	0.574 / 2.56	0.596 / 2.46
8	0.426 / 3.45	0.447 / 3.29

Image 4096 x 4096 (serial 28.500 ms)

Threads	Outer	Collapse(2)
1	32.000 / 0.89	36.500 / 0.78
2	16.500 / 1.73	18.500 / 1.54
4	9.500 / 3.00	9.000 / 3.17
8	6.500 / 4.38	7.500 / 3.80

Image 8192 x 8192 (serial 112.000 ms)

Threads	Outer	Collapse(2)
1	111.000 / 1.01	134.000 / 0.84
2	59.000 / 1.90	68.000 / 1.65
4	31.000 / 3.61	33.000 / 3.39
8	20.000 / 5.60	22.000 / 5.09

Explanation of the speedup. The speedup improves with image size: on 8 threads it is 3.45 for 1024 x 1024 (efficiency 0.43), 4.38 for 4096 x 4096 and 5.60 for 8192 x 8192 (efficiency 0.70). For small images the cost of starting the threads is a bigger part of the short run time, and each pixel needs only 9 multiplications, so the large images are also limited by memory speed. `collapse(2)` is never better than the outer loop alone here: there are far more rows than threads, and it is slower on 1 thread (for example 0.84 against 1.01 at 8192), probably because of the extra index arithmetic needed to split the merged loop. This run was done once, so differences of a few percent are within noise.

7. Overall conclusions

- Programs that do little work per memory access (search, transpose, convolution) reach about 4 to 7 times on 8 threads at best, and only for large inputs.
- Parallel search cannot stop early like serial search, so it is slower when the key is near the start of the array.
- Parallel odd-even sort needs a barrier after every phase, and with N phases of small work this cost outweighs the gain: it did not beat the serial version of the same algorithm for any size tested.

- For histograms, sharing the counters with `atomic` or `critical` is correct but slower than serial code. Private per-thread counters (local copy or `reduction`) are both correct and fast.
- Memory access order matters as much as parallelism: the blocked transpose was faster than the plain version at the same thread count for the large matrices.